An Expository Report on Q1: A Crepant Double-Octic Model, Point Counts, Rigidity, and Classification

Prepared from the Q1 project data

July 13, 2026

Abstract

This report rewrites the Q1 analysis in an expository style. The intended reader is an advanced undergraduate who has seen projective varieties, blow-ups, and basic finite fields, but not necessarily the special vocabulary of double-octic Calabi–Yau threefolds. The main object is a birational double-octic model

$$Y_0 : \quad R^2 = \Delta(a, b, c, d) \subset \mathbb{P}(1, 1, 1, 1, 4),$$

obtained by projecting a quintic threefold from a triple point. The report explains the relevant background, derives the double-cover equation, states the crepantness test, gives the complete center ledger needed for the resolution, and records the projective small resolutions of the final four ordinary double points.

The corrected arithmetic conclusion is that the smooth projective crepant model X has

$$h^{1,1}(X) = 66, \qquad h^{2,1}(X) = 0, \qquad e(X) = 132.$$

For good primes $p \notin \{2, 3, 13\}$,

$$\#X(\mathbb{F}_p) = 1 + p^3 + 66(p^2 + p) - a_p,$$

where a_p agrees with the coefficient of the rational weight-four newform of level 13, LMFDB label 13.4.a.a. The accompanying files in this folder contain Python and Macaulay2 code which regenerate the resolution ledger, verify the singular-locus and divisor checks, and recompute the finite-field counts used in the report.

Contents

Chapter 1

What Is Being Proved

1.1 Main Statement

Let $Q_1 \subset \mathbb{P}^4$ be the projective quintic candidate whose affine equation is displayed in Chapter 3. Projection from the triple point

$$[1 : 1 : 0 : 1 : 0]$$

gives a double cover of $\mathbb{P}^3$. In coordinates $[a : b : c : d]$ on the target $\mathbb{P}^3$, this double cover is

$$Y_0 : \quad R^2 = \Delta(a, b, c, d),$$

where Δ is an octic homogeneous polynomial. The double cover is singular because the branch octic is singular. The goal is to resolve those singularities without changing the canonical class.

Theorem 1.1 (Q1 crepant double-octic model)**.** *The Q1 double cover Y_0 admits a smooth projective crepant resolution*

$$\pi : X \longrightarrow Y_0.$$

The resolution is obtained by the ordered divisorial center sequence in Table 5.1, followed by projective small resolutions of four ordinary double points using the global Weil divisors in Table 5.2. The final threefold X is a rigid Calabi–Yau threefold over $\mathbb{Q}$ with

$$h^{1,1}(X) = 66, \qquad h^{2,1}(X) = 0, \qquad e(X) = 132.$$

For every good prime $p \notin \{2, 3, 13\}$,

$$\#X(\mathbb{F}_p) = 1 + p^3 + 66(p^2 + p) - a_p(13.4.a.a).$$

Thus the two-dimensional middle Galois representation of X is identified with the representation attached to the rational weight-four newform of level 13, away from the bad Euler factors.

The result has three logically separate parts. First, the singular double cover must be resolved crepantly. Second, the resulting smooth model must have the numerical invariants stated above. Third, the finite-field traces must identify the modular form and the classification entry. The report keeps these parts separate because mixing them is a common source of false positives in double-octic calculations.

1.2 Corrections to Earlier Text

The existing PDF report contains useful material, but it also contains errors that must not be carried forward. This section highlights the erroneous text in red and gives the correction.

1.2.1 The $p = 107$ Count

The following row is wrong:

$$\#Y_0(\mathbb{F}_{107}) = 2027154, \qquad 58 \cdot 107^2 + 82 \cdot 107 = 672846, \qquad \#X(\mathbb{F}_{107}) = 2700000.$$

There are two independent errors. First,

$$58 \cdot 107^2 + 82 \cdot 107 = 58 \cdot 11449 + 8774 = 664042 + 8774 = 672816,$$

not 672846. Second, the singular double-cover count recomputed by `report/code/q1_point_counts.py` is

$$\#Y_0(\mathbb{F}_{107}) = 1315608.$$

Therefore the corrected resolved count is

$$\#X(\mathbb{F}_{107}) = 1315608 + 672816 = 1988424.$$

The trace extraction then gives

$$\begin{aligned} a_{107} &= 1 + 107^3 + 66(107^2 + 107) - 1988424 \\ &= 1 + 1225043 + 66(11556) - 1988424 \\ &= 1987740 - 1988424 \\ &= -684. \end{aligned}$$

This corrected value is exactly the additional coefficient needed in the enlarged Livne test set.

1.2.2 The Repository K3-Fibration Octic

The following inference is wrong:

The repository file `data/Q1/K3_Fibration.m2` may be used as the same projective Q1 branch octic.

The comparison script `report/code/q1_repository_comparison.m2` decomposes the singular locus of both octics. The paper model has seven singular lines and one singular point. The repository K3-fibration octic has eight singular lines and one singular point. Projective singular-locus component signatures are invariant under projective changes of coordinates, so these two octics are not projectively isomorphic. Any K3-fibration data extracted from the repository octic must therefore be treated as a separate calculation unless an additional birational or derived comparison is supplied.

1.3 Reproducibility Files

All new files relevant to this rewrite are in the `report/` folder. The main files are:

- `q1_expository_report.tex`: this LaTeX source.
- `q1_expository_report.pdf`: the compiled report.
- `q1_references.bib`: bibliography.
- `code/q1_crepant_ledger.py`: regenerates the center ledger and table fragments.
- `code/q1_point_counts.py`: recomputes singular counts, crepant corrections, resolved counts, and traces.
- `code/q1_model_checks.m2`: checks the branch singular-locus decomposition and the global small-resolution divisors.
- `code/q1_repository_comparison.m2`: checks the non-isomorphism warning for the older repository octic.
- `data/`: generated CSV and JSON data.
- `tables/`: generated LaTeX table fragments.

Chapter 2

Background

2.1 Double Covers and Branch Divisors

A double cover is a map $X \to Y$ whose general fiber has two points. In the simplest affine situation it is written

$$R^2 = f(y_1, \dots, y_n).$$

The divisor $B = (f = 0) \subset Y$ is the branch divisor. Away from B, the two points above a base point are $R = \sqrt{f}$ and $R = -\sqrt{f}$. Above B, these two points collide.

For a global projective double cover one usually chooses a line bundle L on Y and a section

$$f \in H^0(Y, L^{\otimes 2}).$$

Then $B = (f = 0)$ is a divisor in the linear system $|2L|$. The canonical class of the double cover satisfies

$$K_X = \pi^*(K_Y + L).$$

In this report $Y = \mathbb{P}^3$, $L = \mathcal{O}_{\mathbb{P}^3}(4)$, and B is an octic surface. Since

$$K_{\mathbb{P}^3} = \mathcal{O}_{\mathbb{P}^3}(-4),$$

we get

$$K_{\mathbb{P}^3} + L = \mathcal{O}_{\mathbb{P}^3}.$$

Thus a smooth double cover of $\mathbb{P}^3$ branched over a smooth octic has trivial canonical class. Such objects are called double-octic Calabi–Yau threefolds. The Q1 branch octic is not smooth, so the main problem is to resolve the singular double cover without changing the canonical class. This is the crepant resolution problem.

Double-octic Calabi–Yau threefolds have been studied extensively by Cynk, Meyer, Szemberg, van Straten, and others [2–5]. The crepant-resolution criterion used here is the same elementary canonical-class computation that underlies the Cynk–Hulek and Ingalls–Logan criteria for double covers [7].

2.2 Crepant Resolutions

Let X be a singular variety with canonical divisor K_X, and let

$$\pi : \widetilde{X} \to X$$

be a resolution of singularities. The resolution is crepant if

$$K_{\widetilde{X}} = \pi^* K_X.$$

This means the resolution introduces no discrepancy in the canonical divisor. For Calabi–Yau geometry this is the right notion of resolution: if K_X is trivial and π is crepant, then $K_{\widetilde{X}}$ is also trivial.

For double covers, it is often easier to blow up the base Y and then normalize the pulled-back double cover. Suppose Y is smooth, $B \subset Y$ is a branch divisor in $|2L|$, and $Z \subset B$ is a smooth center of codimension c. Let

$$\sigma : Y' \to Y$$

be the blow-up of Z, with exceptional divisor E. If $m = \mathrm{mult}_Z(B)$, then the normalized transformed branch divisor is

$$B' = \sigma^* B - 2 \left\lfloor \frac{m}{2} \right\rfloor E,$$

and the transformed line bundle is

$$L' = \sigma^* L - \left\lfloor \frac{m}{2} \right\rfloor E.$$

The canonical divisor of a blow-up satisfies

$$K_{Y'} = \sigma^* K_Y + (c-1)E.$$

Therefore

$$K_{Y'} + L' = \sigma^*(K_Y + L) + \left(c - 1 - \left\lfloor \frac{m}{2} \right\rfloor \right) E.$$

The blow-up is crepant for the double cover exactly when

$$c - 1 - \left\lfloor \frac{m}{2} \right\rfloor = 0. \tag{2.1}$$

For Q1, every divisorial center is one of the following two types:

$$(c, m) = (2, 2) \quad \text{or} \quad (c, m) = (3, 4).$$

The computations are explicit:

$$2 - 1 - \left\lfloor \frac{2}{2} \right\rfloor = 1 - 1 = 0,$$

and

$$3 - 1 - \left\lfloor \frac{4}{2} \right\rfloor = 2 - 2 = 0.$$

Thus every center in the ledger is crepant.

2.3 Ordinary Double Points and Small Resolutions

After the divisorial blow-ups, Q1 still has four ordinary double points. A three-dimensional ordinary double point has local analytic equation

$$AB = LK.$$

It has two small resolutions. For example, blowing up the ideal (A, L) gives the charts

$$A = L\lambda, \qquad B = \lambda K,$$

and

$$L = A\mu, \qquad B = \mu K.$$

Both charts are smooth. The exceptional set is a copy of $\mathbb{P}^1$, not a divisor, so the resolution is small. Small resolutions of nodes are crepant [1].

The subtle point is projectivity. Local small resolutions can be chosen independently in analytic neighborhoods, but an arbitrary collection of choices need not come from a projective global variety. In Q1, projectivity is supplied by three global Weil divisors on the double cover. Blowing up those divisors realizes the required local small resolutions at the four nodes and is a projective operation globally.

2.4 Point Counts and Traces

Let X be a smooth projective Calabi–Yau threefold over $\mathbb{Q}$, and suppose p is a good prime. The Lefschetz trace formula expresses $\#X(\mathbb{F}_p)$ as an alternating sum of Frobenius traces on etale cohomology. For a rigid Calabi–Yau threefold with all divisors defined over $\mathbb{Q}$, the formula has the form

$$\#X(\mathbb{F}_p) = 1 + p^3 + h^{1,1}(p + p^2) - a_p,$$

where a_p is the trace of Frobenius on the two-dimensional middle cohomology H^3. In this report $h^{1,1} = 66$, so

$$a_p = 1 + p^3 + 66(p + p^2) - \#X(\mathbb{F}_p). \tag{2.2}$$

The finite-field count therefore turns geometry into a sequence of integers a_p. That sequence can then be compared with Fourier coefficients of modular forms.

Chapter 3

The Q1 Model

3.1 The Affine Quintic

The Q1 affine equation is

$$\begin{aligned} f = {} & y^2z^3 + xy^2zw - 2xyz^2w + y^2z^2w - x^2yw^2 + x^2zw^2 - 2xyzw^2 + x^2w^3 \\ & + xy^2z + xyz^2 - 3y^2z^2 - yz^3 - x^2zw + 3xyzw - 2y^2zw + xz^2w \\ & - x^2w^2 + 2xyw^2 + yzw^2 - xw^3 - xy^2 - 2xyz + 2y^2z + 2yz^2 \\ & + x^2w - xyw - xzw + xy - yz. \end{aligned}$$

Homogenizing this equation in $\mathbb{P}^4$ gives a quintic Q_1. The point

$$p = [1 : 1 : 0 : 1 : 0]$$

is a triple point. Projection from this point is the geometric operation that produces the double-octic model.

3.2 Projection from the Triple Point

Write the projection substitution as

$$X_0 = u + a, \qquad X_1 = u + b, \qquad X_2 = c, \qquad X_3 = u, \qquad X_4 = d.$$

After substituting into the quintic equation, the result is quadratic in u:

$$F(u + a, u + b, c, u, d) = u^2q_3 + uq_4 + q_5.$$

Here

$$\begin{aligned} q_3 &= ad(b + c - d), \\ q_4 &= a^2bc + a^2bd + a^2c^2 - a^2cd + ab^2d - abc^2 + abcd - 2abd^2 \\ &\quad - 2ac^2d + 3acd^2 - ad^3 + bc^2d - 2bcd^2 + bd^3, \end{aligned}$$

and

$$\begin{aligned} q_5 = {} & a^3bc + a^2b^2d - a^2bc^2 - a^2bcd - ab^2d^2 + 2abcd^2 \\ & - abd^3 - b^2cd^2 + b^2d^3. \end{aligned}$$

The discriminant of the quadratic is

$$\Delta = q_4^2 - 4q_3q_5. \tag{3.1}$$

Thus the function field extension obtained from the projection is generated by

$$R = 2q_3u + q_4.$$

Indeed,

$$R^2 = (2q_3u + q_4)^2 = q_4^2 - 4q_3q_5 = \Delta$$

after using $q_3u^2 + q_4u + q_5 = 0$. Conversely, on the open set $q_3 \neq 0$,

$$u = \frac{-q_4 + R}{2q_3}.$$

Therefore the projection is birational to the double cover

$$Y_0 : \quad R^2 = \Delta(a, b, c, d) \subset \mathbb{P}(1, 1, 1, 1, 4).$$

3.3 Why the Ambient Space Is Weighted

The variables a, b, c, d are homogeneous coordinates of weight 1 on $\mathbb{P}^3$. The polynomial Δ is homogeneous of degree 8. Thus R must have weight 4 for R^2 to have degree 8. This is why the double cover is naturally embedded in

$$\mathbb{P}(1, 1, 1, 1, 4).$$

The branch surface is the octic

$$B = (\Delta = 0) \subset \mathbb{P}^3.$$

The singularities of Y_0 are controlled by the singularities of B.

3.4 Integral Model and Bad Primes

The displayed q_3, q_4, q_5 have integral coefficients, so $\Delta \in \mathbb{Z}[a, b, c, d]$. The transformation $R = 2q_3u + q_4$ uses the factor 2, so the natural integral double-cover model is over $\mathbb{Z}[1/2]$. The report uses the conservative bad set

$$S = \{2, 3, 13\}.$$

The prime 2 is forced by the double-cover construction. The prime 13 is the level of the modular form. The prime 3 is included because the endpoint reduction is anomalous for the generic good-prime correction, and the Faltings–Serre–Livne comparison must not use it as a good prime.

Chapter 4

Branch Geometry

4.1 The Singular Locus of the Branch Octic

The Macaulay2 script `report/code/q1_model_checks.m2` decomposes the singular locus of $B = (\Delta = 0)$. It finds eight components: seven projective lines and one projective point. The seven lines are:

$$\begin{aligned} &L_1 :\ a = b = 0, && L_2 :\ a - b = c = 0, \\ &L_3 :\ a = d = 0, && L_4 :\ b = c - d = 0, \\ &L_5 :\ a = c - d = 0, && L_6 :\ c = d = 0, \\ &L_7 :\ a - d = b + c - d = 0. \end{aligned}$$

The isolated singular point has ideal

$$c - 3d = 0, \qquad b + 2d = 0, \qquad a - 4d = 0.$$

In homogeneous coordinates this is the point $[4 : -2 : 3 : 1]$, or equivalently $[1 : -1/2 : 3/4 : 1/4]$ over $\mathbb{Q}$. The singular-locus output from Macaulay2 is:

component	projective dimension	degree
$L_1, \ldots, L_7$	1	1
isolated point	0	1.

4.2 Transverse Types

At a general point of a singular branch line, the double cover looks like a family of surface singularities over a line. The transverse types recorded in the Q1 ledger are:

line	generic transverse type	number of curve blow-ups
L_1	cA_8	4
L_2	cA_4	2
L_3	cA_7	4
L_4	cA_8	4
L_5	cA_1	1
L_6	cA_7	4
L_7	cA_1	1.

The number of curve blow-ups in the third column is the length of the admissible double-cover resolution chain along that line. Adding these gives

$$4 + 2 + 4 + 4 + 1 + 4 + 1 = 20. \tag{4.1}$$

These are the twenty "old branch line chain" blow-ups in the resolution ledger.

4.3 Special Points

The line arrangement has special intersection points. Six of them become branch-multiplicity-four point centers at the moment they are blown up. The center IDs are

$$P13, \quad P145, \quad P27, \quad P356, \quad P46, \quad P47.$$

Each is a point in the base, hence has codimension 3. Each has branch multiplicity 4. Therefore all six point blow-ups satisfy

$$3 - 1 - \left\lfloor \frac{4}{2} \right\rfloor = 0.$$

This is the second main numerical input for crepantness.

Chapter 5

The Crepant Resolution

5.1 The Complete Divisorial Ledger

The base center ledger has 23 rows, some with repeats. Expanding the repeats gives 36 actual divisorial blow-ups:

20 line-chain blow-ups+6 point blow-ups+5 exceptional curve blow-ups+1 Γ_{46} main blow-up+4 Γ_{46} endpoi (5.1)

The explicit affine chart count is

$$20 \cdot 2 + 6 \cdot 3 + 5 \cdot 2 + 1 \cdot 2 + 4 \cdot 2 = 40 + 18 + 10 + 2 + 8 = 78. \tag{5.2}$$

Every row has discrepancy term 0. Table 5.1 is generated by `q1_crepant_ledger.py`.

Table 5.1: Base resolution centers for Q1. Repeated centers represent ordered cA-chain blow-ups.

Center	Stage	Kind	Equations	Repeat	Codim.	Mult.	Disc.
L1	old branch line chains	curve	$\langle a, b\rangle$	4	2	2	0
L2	old branch line chains	curve	$\langle a-b, c\rangle$	2	2	2	0
L3	old branch line chains	curve	$\langle a, d\rangle$	4	2	2	0
L4	old branch line chains	curve	$\langle b, c-d\rangle$	4	2	2	0
L5	old branch line chains	curve	$\langle a, c-d\rangle$	1	2	2	0
L6	old branch line chains	curve	$\langle c, d\rangle$	4	2	2	0
L7	old branch line chains	curve	$\langle a-d, b+c-d\rangle$	1	2	2	0
P13	multiplicity-four points	point	$\langle a, b, d\rangle$	1	3	4	0
P145	multiplicity-four points	point	$\langle a, b, c-d\rangle$	1	3	4	0
P27	multiplicity-four points	point	$\langle a-d, b+c-d, c\rangle$	1	3	4	0
P356	multiplicity-four points	point	$\langle a, c, d\rangle$	1	3	4	0

Center	Stage	Kind	Equations	Repeat	Codim.	Mult.	Disc.
P46	multiplicity-four points	point	$\langle b, c-d, c\rangle$	1	3	4	0
P47	multiplicity-four points	point	$\langle b, c-d, a-d\rangle$	1	3	4	0
E145_x	exceptional curves after point blow-ups	curve	$\langle x, exceptional\_145\rangle$	1	2	2	0
E145_z	exceptional curves after point blow-ups	curve	$\langle z, exceptional\_145\rangle$	1	2	2	0
E356_x	exceptional curves after point blow-ups	curve	$\langle x, exceptional\_356\rangle$	1	2	2	0
E356_z	exceptional curves after point blow-ups	curve	$\langle z, exceptional\_356\rangle$	1	2	2	0
E46_q	exceptional curves after point blow-ups	curve	$\langle q, exceptional\_46\rangle$	1	2	2	0
Gamma_46	residual Gamma_46	curve	$\langle e, (T-1)*(r-e)+T\rangle$	1	2	2	0
Gamma_46_T1_a	Gamma_46 endpoint T=1	curve	$\langle n, tau\rangle$	1	2	2	0
Gamma_46_T1_b	Gamma_46 endpoint T=1	curve	$\langle m, tau\rangle$	1	2	2	0
Gamma_46_Tinf_a	Gamma_46 endpoint T=infinity	curve	$\langle e, K\rangle$	1	2	2	0
Gamma_46_Tinf_b	Gamma_46 endpoint T=infinity	curve	$\langle m, U\rangle$	1	2	2	0

5.2 How to Read the Ledger

Each row of Table 5.1 records:

- the center ID;
- the stage of the construction;
- whether the center is a curve or a point;
- local equations cutting out the center;
- the number of repeated ordered blow-ups;
- the codimension c;
- the branch multiplicity m;
- the discrepancy term $c - 1 - \lfloor m/2 \rfloor$.

For example, L_1 is the line $\langle a, b\rangle$. It is repeated four times because the generic transverse singularity is cA_8. Each occurrence is a curve blow-up with $c = 2$ and $m = 2$, so it is crepant. The point $P13$ has local equations $\langle a, b, d\rangle$, codimension 3, and branch multiplicity 4, so it is crepant by the point computation.

5.3 The Residual Curve Γ_{46}

The most delicate chart is the $P46$ endpoint chart. After previous blow-ups, a residual branch curve remains:

$$\Gamma_{46}: \quad e = 0, \qquad N = (T-1)(r-e) + T = 0.$$

This is a curve center of branch multiplicity 2. The main blow-up of $e = N = 0$ is crepant. Compactifying the T-line produces two finite endpoint blow-ups at $T = 1$,

$$n = \tau = 0, \qquad m = \tau = 0,$$

and two infinite endpoint blow-ups at $T = \infty$,

$$e = K = 0, \qquad m = U = 0.$$

All four endpoint centers are curves with branch multiplicity 2. Thus each has discrepancy

$$2 - 1 - \left\lfloor \frac{2}{2} \right\rfloor = 0.$$

The endpoint chains matter arithmetically. Without them the point-count correction has the wrong endpoint coefficient, and the $p = 107$ row does not agree with the level-13 modular form.

5.4 Final Ordinary Double Points

After the 36 divisorial blow-ups, the only remaining singularities are four ordinary double points:

$$N13\_0, \qquad N13\_\text{half}, \qquad N46\_0, \qquad N46\_qt.$$

They are resolved by the three global Weil divisors in Table 5.2. This table is generated by `q1_crepant_ledger.py`.

Table 5.2: Global Weil divisors used for the projective small resolutions.

Divisor	Equations	Nodes resolved
`Dab_plus`	$\langle b - a, R - a * c * (a^2 + c * d - d^2)\rangle$	N13_0
`Db_minus`	$\langle b, R + a * (c - d) * (a * c - 2 * c * d + d^2)\rangle$	N13_half, N46_0
`Dd_minus`	$\langle d, R + a * c * (a * b + a * c - b * c)\rangle$	N46_qt

The Macaulay2 script `q1_model_checks.m2` verifies that each divisor lies on the double cover:

divisor	lies on $R^2 = \Delta$
Dab_plus	`true`
Db_minus	`true`
Dd_minus	`true`.

Because these are global divisors, their blow-ups are projective. Away from the nodes they are Cartier and the blow-up is an isomorphism; at the nodes they realize the standard small resolution.

5.5 Conclusion of the Crepant Resolution

Proposition 5.1. *The resolution $X \to Y_0$ obtained from the ledger and small resolutions is projective and crepant.*

Proof. Every divisorial center is a smooth closed center on a projective partial base. Blowing up such a center is projective. The transformed double cover is normalized using the standard rule

$$B' = \sigma^* B - 2 \left\lfloor \frac{m}{2} \right\rfloor E.$$

For every curve center in the ledger, $(c, m) = (2, 2)$, so the discrepancy term is 0. For every point center, $(c, m) = (3, 4)$, so the discrepancy term is 0. Hence all divisorial steps are crepant.

The remaining four singularities are ordinary double points. The three global Weil divisors in Table 5.2 give projective small resolutions of those nodes. Small resolutions of ordinary double points are crepant because no exceptional divisor is introduced. Therefore the full map $X \to Y_0$ is projective and crepant. □

Chapter 6

Finite-Field Counts

6.1 Counting the Singular Double Cover

Let p be an odd prime. A point of $\mathbb{P}^3(\mathbb{F}_p)$ is represented uniquely by one of

$$(1, b, c, d), \qquad (0, 1, c, d), \qquad (0, 0, 1, d), \qquad (0, 0, 0, 1).$$

At such a point P, the fiber of $Y_0 \to \mathbb{P}^3$ is the equation

$$R^2 = \Delta(P).$$

If χ is the quadratic character of $\mathbb{F}_p$, with $\chi(0) = 0$, then the number of R-solutions is

$$1 + \chi(\Delta(P)).$$

Thus

$$\#Y_0(\mathbb{F}_p) = \sum_{P \in \mathbb{P}^3(\mathbb{F}_p)} \left(1 + \chi(\Delta(P))\right). \tag{6.1}$$

This is exactly what `q1_point_counts.py` computes. The script uses the compact formula

$$\Delta = q_4^2 - 4q_3q_5$$

instead of the expanded octic, which reduces the risk of transcription errors.

6.2 The Crepant Correction

The resolution ledger gives a uniform correction for good primes:

$$\#X(\mathbb{F}_p) = \#Y_0(\mathbb{F}_p) + 58p^2 + 82p. \tag{6.2}$$

The coefficient 58 comes from the divisor and exceptional-surface part of the resolution ledger. The coefficient 82 records the lower-dimensional exceptional fibers, including the endpoint corrections and the four small-resolution curves. Formula (6.2) is a finite-field version of the same exceptional-stratum bookkeeping used to compute the Picard and Euler data.

For example, at $p = 5$,

$$58p^2 + 82p = 58 \cdot 25 + 82 \cdot 5 = 1450 + 410 = 1860.$$

The singular count is 253, hence

$$\#X(\mathbb{F}_5) = 253 + 1860 = 2113.$$

The trace is

$$\begin{aligned} a_5 &= 1 + 5^3 + 66(5^2 + 5) - 2113 \\ &= 1 + 125 + 66 \cdot 30 - 2113 \\ &= 2106 - 2113 \\ &= -7. \end{aligned}$$

6.3 Computed Count Table

The following table is generated from `report/data/q1_point_counts_extended.csv`. The primes 3 and 13 are shown for audit purposes but are not good primes in the final comparison.

Table 6.1: Finite-field counts for the Q1 singular double octic and its crepant model.

p	$\#Y_0(\mathbb{F}_p)$	$58p^2 + 82p$	$\#X(\mathbb{F}_p)$	$a_p(X)$	Use
3	62	768	830	-10	bad
5	253	1860	2113	-7	good
7	637	3416	4053	-13	good
11	2150	7920	10070	-26	good
13	3329	10868	14197	13	bad
17	6877	18156	25033	77	good
19	9570	22496	32066	-126	good
23	16128	32568	48696	-96	good
29	30736	51156	81892	-82	good
31	36788	58280	95068	196	good
37	61145	82436	143581	-131	good
41	81378	100860	182238	336	good
43	93813	110768	204581	-201	good
47	120849	131976	252825	-105	good
53	170934	167268	338202	-432	good
59	232578	206736	439314	-294	good
61	255830	220820	476650	-56	good
67	335126	265856	600982	478	good
71	397095	298200	695295	9	good
73	430384	315068	745452	98	good
79	540400	368456	908856	1304	good
83	625880	406368	1032248	-308	good

p	$\#Y_0(\mathbb{F}_p)$	$58p^2+82p$	$\#X(\mathbb{F}_p)$	$a_p(X)$	Use
89	768104	466716	1234820	-1190	good
97	986324	553676	1540000	70	good
101	1109874	599940	1709814	420	good
103	1175364	623768	1799132	588	good
107	1315608	672816	1988424	-684	good
109	1387961	698036	2085997	373	good

6.4 The Corrected $p = 107$ Row

The $p = 107$ row is the best audit test because it is the extra prime needed by the enlarged bad set. The computation is:

$$\#Y_0(\mathbb{F}_{107}) = 1315608,$$

$$58 \cdot 107^2 + 82 \cdot 107 = 672816,$$

and therefore

$$\#X(\mathbb{F}_{107}) = 1988424.$$

Using (2.2),

$$\begin{aligned} a_{107} &= 1 + 107^3 + 66(107^2 + 107) - 1988424 \\ &= -684. \end{aligned}$$

This is the coefficient of the level-13 newform at 107. The old 2700000 row would give an impossible trace for the asserted modular form and is therefore rejected.

Chapter 7

Rigidity and Hodge Numbers

7.1 Picard Rank

The geometric Picard calculation in the Q1 ledger gives

$$\rho(X) = 66.$$

The count is not inferred from the modular form. It is computed from the divisor ledger of the resolution. The important distinction is that the four final small resolutions add curves but no divisors. The endpoint chains add four divisors, taking the divisor ledger from 62 to 66.

The report uses

$$h^{1,1}(X) = \rho(X) = 66.$$

The equality is justified by the fact that the divisor classes in the constructed projective model account for the algebraic H^2 contribution appearing in the point-count formula at good primes.

7.2 Euler Characteristic

The Euler characteristic recorded by the completed resolution ledger is

$$e(X) = 132.$$

For a smooth Calabi–Yau threefold,

$$e(X) = 2(h^{1,1}(X) - h^{2,1}(X)). \tag{7.1}$$

Substituting $e(X) = 132$ and $h^{1,1}(X) = 66$ gives

$$132 = 2(66 - h^{2,1}(X)).$$

Dividing by 2,

$$66 = 66 - h^{2,1}(X),$$

so

$$h^{2,1}(X) = 0.$$

This is the promised rigidity computation.

7.3 Relation with Branch Deformations

The deformation-theoretic viewpoint gives the same conclusion. A first-order deformation of a double cover of $\mathbb{P}^3$ branched over an octic is represented by

$$\Delta + \varepsilon G,$$

where G is another homogeneous octic. The vector space of homogeneous octics in four variables has dimension

$$\binom{8+3}{3} = \binom{11}{3} = 165.$$

The fixed-center deformation calculation imposes the same branch multiplicity conditions along:

$$L_1, \dots, L_7, \quad P13, \dots, P47, \quad E145_x, E145_z, E356_x, E356_z, E46_q, \quad \Gamma_{46},$$

and the endpoint tangent conditions. The project data record the rank reductions as follows:

stage	remaining dimension
homogeneous octics	165
original branch lines and six point centers	25
plus five exceptional curve centers	9
plus endpoint tangent directions	2
plus Γ_{46} transverse condition	1.

The final one-dimensional space is generated by Δ itself. Scaling the defining equation does not change the branch divisor in projective space, so no true deformation remains. This agrees with the Hodge-number computation above and with the double-cover deformation framework of Cynk and van Straten [5].

Theorem 7.1 (Rigidity)**.** *The smooth projective crepant model X is rigid:*

$$h^{2,1}(X) = 0.$$

Proof. The resolution ledger gives $h^{1,1} = 66$ and $e = 132$. Equation (7.1) for Calabi–Yau threefolds then forces $h^{2,1} = 0$. Equivalently, the fixed-center deformation calculation leaves only the scalar multiple of the branch equation, which is projectively trivial. □

Chapter 8

Modularity and Classification

8.1 Trace Sequence

At good primes the trace sequence begins

p	5	7	11	17	19	23	29	31
$a_p(X)$	-7	-13	-26	77	-126	-96	-82	196.

These traces match the rational weight-four newform of level 13, LMFDB label 13.4.a.a [12]. The same level-13 form appears in Logan's graph-hypersurface work, where it is called the form 13/1 in Meyer's notation [10].

8.2 Bad Set and Livne Test

The comparison uses the bad set

$$S = \{2, 3, 13\}.$$

The residual mod-2 representation is encoded by the cubic

$$g(x) = x^3 - 23x^2 - 9x - 1.$$

Its discriminant is

$$\operatorname{disc}(g) = -6656 = -2^9 \cdot 13.$$

Thus the residual cubic is unramified away from 2 and 13. The prime 3 is included because of the endpoint reduction, not because of the residual cubic.

With $S = \{2, 3, 13\}$, the quadratic character quotient has basis

$$-1, \quad 2, \quad 3, \quad 13.$$

It is a four-dimensional $\mathbb{F}_2$-space, so a non-cubic Livne comparison must test all $2^4 - 1 = 15$ nonzero vectors. The missing vector in the smaller bad-set calculation is realized at $p = 107$. This is why the corrected $p = 107$ point count is essential.

The Faltings–Serre–Livne method then compares two two-dimensional ℓ-adic Galois representations by checking determinant, residual representation, and a finite set of Frobenius traces [6, 9, 11]. Here the determinants are p^3, and the traces match the level-13 form on the completed test set.

8.3 Classification of the Threefold

The invariants used for classification are

$$(h^{1,1}, h^{2,1}) = (66, 0), \qquad e = 132,$$

and the modular form is

$$f = 13.4.a.a.$$

Thus Q1 is a rigid Calabi–Yau threefold realizing the level-13 weight-four motive. It is not classified merely by the modular form: different rigid Calabi–Yau threefolds can realize the same two-dimensional motive. Logan's construction realizes the same level-13 form but has a different Picard number [10]. The Q1 model is therefore a same-form, different-topology realization in the project catalogue.

8.4 K3-Fibration Data

Some earlier notes discuss a K3 pencil through the branch line L_5. K3 degenerations are normally organized by Kulikov types I, II, and III [8]. The caution in Chapter 1 applies here: the repository file `data/Q1/K3_Fibration.m2` is not projectively isomorphic to the paper double-octic model. Therefore the K3-fibration calculations from that repository octic should be cited only as conditional or separate data unless a comparison map is supplied. The crepant resolution, point counts, rigidity, and classification in this report do not require that K3-fibration section.

Chapter 9

Reproducibility

9.1 Python Ledger Script

The script `code/q1_crepant_ledger.py` records the center list and verifies:

$$\text{divisorial blow-ups} = 36, \qquad \text{affine charts} = 78, \qquad \text{all discrepancy terms} = 0.$$

It also writes the center tables used in this report.

9.2 Python Point-Count Script

The script `code/q1_point_counts.py` implements:

$$\#Y_0(\mathbb{F}_p) = \sum_{P \in \mathbb{P}^3(\mathbb{F}_p)} (1 + \chi(\Delta(P))),$$

$$\#X(\mathbb{F}_p) = \#Y_0(\mathbb{F}_p) + 58p^2 + 82p,$$

and

$$a_p = 1 + p^3 + 66(p^2 + p) - \#X(\mathbb{F}_p).$$

It verifies the known regression rows

$$p = 3, 5, 7, 11, 107$$

and can recompute the extended table through 109.

9.3 Macaulay2 Scripts

The script `code/q1_model_checks.m2` verifies that the paper branch singular locus has seven lines and one point, and that the three global small-resolution divisors lie on the double cover. The script `code/q1_repository_comparison.m2` verifies the model mismatch warning by comparing singular-locus component signatures.

Appendix A

Expanded Center List

The following table expands the repeated line-chain centers. It is useful when auditing the actual order of the 36 divisorial blow-ups.

Table A.1: Expanded ordered list of the 36 divisorial crepant blow-ups.

Order	Occurrence	Stage	Equations	Codim.	Mult.	Disc.	Charts
1	`L1_1`	old branch line chains	$\langle a; b\rangle$	2	2	0	2
2	`L1_2`	old branch line chains	$\langle a; b\rangle$	2	2	0	2
3	`L1_3`	old branch line chains	$\langle a; b\rangle$	2	2	0	2
4	`L1_4`	old branch line chains	$\langle a; b\rangle$	2	2	0	2
5	`L2_1`	old branch line chains	$\langle a-b; c\rangle$	2	2	0	2
6	`L2_2`	old branch line chains	$\langle a-b; c\rangle$	2	2	0	2
7	`L3_1`	old branch line chains	$\langle a; d\rangle$	2	2	0	2
8	`L3_2`	old branch line chains	$\langle a; d\rangle$	2	2	0	2
9	`L3_3`	old branch line chains	$\langle a; d\rangle$	2	2	0	2
10	`L3_4`	old branch line chains	$\langle a; d\rangle$	2	2	0	2
11	`L4_1`	old branch line chains	$\langle b; c-d\rangle$	2	2	0	2
12	`L4_2`	old branch line chains	$\langle b; c-d\rangle$	2	2	0	2
13	`L4_3`	old branch line chains	$\langle b; c-d\rangle$	2	2	0	2

Order	Occurrence	Stage	Equations	Codim.	Mult.	Disc.	Charts
14	L4_4	old branch line chains	$\langle b; c-d\rangle$	2	2	0	2
15	L5	old branch line chains	$\langle a; c-d\rangle$	2	2	0	2
16	L6_1	old branch line chains	$\langle c; d\rangle$	2	2	0	2
17	L6_2	old branch line chains	$\langle c; d\rangle$	2	2	0	2
18	L6_3	old branch line chains	$\langle c; d\rangle$	2	2	0	2
19	L6_4	old branch line chains	$\langle c; d\rangle$	2	2	0	2
20	L7	old branch line chains	$\langle a-d; b+c-d\rangle$	2	2	0	2
21	P13	multiplicity-four points	$\langle a; b; d\rangle$	3	4	0	3
22	P145	multiplicity-four points	$\langle a; b; c-d\rangle$	3	4	0	3
23	P27	multiplicity-four points	$\langle a-d; b+c-d; c\rangle$	3	4	0	3
24	P356	multiplicity-four points	$\langle a; c; d\rangle$	3	4	0	3
25	P46	multiplicity-four points	$\langle b; c-d; c\rangle$	3	4	0	3
26	P47	multiplicity-four points	$\langle b; c-d; a-d\rangle$	3	4	0	3
27	E145_x	exceptional curves after point blow-ups	$\langle x; exceptional\_145\rangle$	2	2	0	2
28	E145_z	exceptional curves after point blow-ups	$\langle z; exceptional\_145\rangle$	2	2	0	2
29	E356_x	exceptional curves after point blow-ups	$\langle x; exceptional\_356\rangle$	2	2	0	2
30	E356_z	exceptional curves after point blow-ups	$\langle z; exceptional\_356\rangle$	2	2	0	2
31	E46_q	exceptional curves after point blow-ups	$\langle q; exceptional\_46\rangle$	2	2	0	2
32	Gamma_46	residual Gamma_46	$\langle e; (T-1)*(r-e)+T\rangle$	2	2	0	2

Order	Occurrence	Stage	Equations	Codim.	Mult.	Disc.	Charts
33	`Gamma_46_T1_a`	Gamma_46 endpoint T=1	$\langle n; tau \rangle$	2	2	0	2
34	`Gamma_46_T1_b`	Gamma_46 endpoint T=1	$\langle m; tau \rangle$	2	2	0	2
35	`Gamma_46_Tinf_a`	Gamma_46 endpoint T=infinity	$\langle e; K \rangle$	2	2	0	2
36	`Gamma_46_Tinf_b`	Gamma_46 endpoint T=infinity	$\langle m; U \rangle$	2	2	0	2

Appendix B

Selected Arithmetic Computations

B.1 Correction Polynomial

The correction polynomial is

$$C(p) = 58p^2 + 82p.$$

For the displayed audit primes:

p	$58p^2$	$82p$	$C(p)$
5	1450	410	1860
7	2842	574	3416
11	7018	902	7920
107	664042	8774	672816.

B.2 Trace Formula Checks

For $p = 7$:

$$\begin{aligned} a_7 &= 1 + 7^3 + 66(7^2 + 7) - 4053 \\ &= 1 + 343 + 66 \cdot 56 - 4053 \\ &= 4040 - 4053 \\ &= -13. \end{aligned}$$

For $p = 11$:

$$\begin{aligned} a_{11} &= 1 + 11^3 + 66(11^2 + 11) - 10070 \\ &= 1 + 1331 + 66 \cdot 132 - 10070 \\ &= 10044 - 10070 \\ &= -26. \end{aligned}$$

For $p = 107$:

$$\begin{aligned} a_{107} &= 1 + 107^3 + 66(107^2 + 107) - 1988424 \\ &= 1 + 1225043 + 66 \cdot 11556 - 1988424 \\ &= 1987740 - 1988424 \\ &= -684. \end{aligned}$$

Appendix C

Macaulay2 Output Summary

Running

```
M2 -script report/code/q1_model_checks.m2
```

from the repository root gives:

```
Q1_BRANCH_COMPONENT_COUNT|8
Q1_BRANCH_COMPONENT|0|projective_dim=1|degree=1|gens=matrix {{d, c}}
Q1_BRANCH_COMPONENT|1|projective_dim=1|degree=1|gens=matrix {{d, a}}
Q1_BRANCH_COMPONENT|2|projective_dim=1|degree=1|gens=matrix {{c, a-b}}
Q1_BRANCH_COMPONENT|3|projective_dim=1|degree=1|gens=matrix {{c-d, b}}
Q1_BRANCH_COMPONENT|4|projective_dim=1|degree=1|gens=matrix {{c-d, a}}
Q1_BRANCH_COMPONENT|5|projective_dim=1|degree=1|gens=matrix {{b, a}}
Q1_BRANCH_COMPONENT|6|projective_dim=1|degree=1|gens=matrix {{b+c-d, a-d}}
Q1_BRANCH_COMPONENT|7|projective_dim=0|degree=1|gens=matrix {{c-3*d, b+2*d, a-4*d}}
Dab_plus_lies_on_cover|true
Db_minus_lies_on_cover|true
Dd_minus_lies_on_cover|true
```

Running

```
M2 -script report/code/q1_repository_comparison.m2
```

ends with

```
PAPER_COMPONENTS|8
REPO_COMPONENTS|9
SIGNATURE_EQUAL|false
```

which is the computational basis for the red warning about the older repository K3-fibration octic.

Appendix D

Generated Data Listings

D.1 Point-Count CSV

The CSV file below is generated by `code/q1_point_counts.py`. It is the table used in Chapter 6.

```
p,singular_model_count,crepant_correction,resolved_count,trace_a_p,good_prime
3,62,768,830,-10,False
5,253,1860,2113,-7,True
7,637,3416,4053,-13,True
11,2150,7920,10070,-26,True
13,3329,10868,14197,13,False
17,6877,18156,25033,77,True
19,9570,22496,32066,-126,True
23,16128,32568,48696,-96,True
29,30736,51156,81892,-82,True
31,36788,58280,95068,196,True
37,61145,82436,143581,-131,True
41,81378,100860,182238,336,True
43,93813,110768,204581,-201,True
47,120849,131976,252825,-105,True
53,170934,167268,338202,-432,True
59,232578,206736,439314,-294,True
61,255830,220820,476650,-56,True
67,335126,265856,600982,478,True
71,397095,298200,695295,9,True
73,430384,315068,745452,98,True
79,540400,368456,908856,1304,True
83,625880,406368,1032248,-308,True
89,768104,466716,1234820,-1190,True
97,986324,553676,1540000,70,True
101,1109874,599940,1709814,420,True
103,1175364,623768,1799132,588,True
107,1315608,672816,1988424,-684,True
109,1387961,698036,2085997,373,True
```

D.2 Resolution Ledger JSON

The JSON file below is generated by `code/q1_crepant_ledger.py`. It records the summary checks, the small-resolution divisors, and the expanded list of the 36 divisorial blow-ups.

```
{
  "summary": {
```

```
  "base_center_count": 23,
  "divisorial_blowup_count": 36,
  "explicit_affine_chart_count": 78,
  "all_discrepancy_terms": [
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0
  ],
  "small_resolution_count": 4,
  "small_resolution_divisors": [
    {
      "divisor_id": "Dab_plus",
      "equations": [
        "b-a",
        "R-a*c*(a^2+c*d-d^2)"
      ],
      "nodes": [
        "N13_0"
      ]
    },
    {
      "divisor_id": "Db_minus",
      "equations": [
        "b",
        "R+a*(c-d)*(a*c-2*c*d+d^2)"
      ],
      "nodes": [
        "N13_half",
        "N46_0"
      ]
    },
    {
      "divisor_id": "Dd_minus",
      "equations": [
        "d",
        "R+a*c*(a*b+a*c-b*c)"
      ],
      "nodes": [
        "N46_qt"
      ]
    }
  ]
},
"expanded_centers": [
  {
    "order": 1,
    "occurrence_id": "L1_1",
    "base_center_id": "L1",
```

```
    "stage": "old branch line chains",
    "kind": "curve",
    "equations": "a; b",
    "codimension": 2,
    "branch_multiplicity": 2,
    "discrepancy_term": 0,
    "chart_count": 2
  },
  {
    "order": 2,
    "occurrence_id": "L1_2",
    "base_center_id": "L1",
    "stage": "old branch line chains",
    "kind": "curve",
    "equations": "a; b",
    "codimension": 2,
    "branch_multiplicity": 2,
    "discrepancy_term": 0,
    "chart_count": 2
  },
  {
    "order": 3,
    "occurrence_id": "L1_3",
    "base_center_id": "L1",
    "stage": "old branch line chains",
    "kind": "curve",
    "equations": "a; b",
    "codimension": 2,
    "branch_multiplicity": 2,
    "discrepancy_term": 0,
    "chart_count": 2
  },
  {
    "order": 4,
    "occurrence_id": "L1_4",
    "base_center_id": "L1",
    "stage": "old branch line chains",
    "kind": "curve",
    "equations": "a; b",
    "codimension": 2,
    "branch_multiplicity": 2,
    "discrepancy_term": 0,
    "chart_count": 2
  },
  {
    "order": 5,
    "occurrence_id": "L2_1",
    "base_center_id": "L2",
    "stage": "old branch line chains",
    "kind": "curve",
    "equations": "a-b; c",
    "codimension": 2,
    "branch_multiplicity": 2,
    "discrepancy_term": 0,
    "chart_count": 2
  },
  {
    "order": 6,
    "occurrence_id": "L2_2",
    "base_center_id": "L2",
    "stage": "old branch line chains",
    "kind": "curve",
    "equations": "a-b; c",
    "codimension": 2,
    "branch_multiplicity": 2,
    "discrepancy_term": 0,
    "chart_count": 2
  },
```

```
{
  "order": 7,
  "occurrence_id": "L3_1",
  "base_center_id": "L3",
  "stage": "old branch line chains",
  "kind": "curve",
  "equations": "a; d",
  "codimension": 2,
  "branch_multiplicity": 2,
  "discrepancy_term": 0,
  "chart_count": 2
},
{
  "order": 8,
  "occurrence_id": "L3_2",
  "base_center_id": "L3",
  "stage": "old branch line chains",
  "kind": "curve",
  "equations": "a; d",
  "codimension": 2,
  "branch_multiplicity": 2,
  "discrepancy_term": 0,
  "chart_count": 2
},
{
  "order": 9,
  "occurrence_id": "L3_3",
  "base_center_id": "L3",
  "stage": "old branch line chains",
  "kind": "curve",
  "equations": "a; d",
  "codimension": 2,
  "branch_multiplicity": 2,
  "discrepancy_term": 0,
  "chart_count": 2
},
{
  "order": 10,
  "occurrence_id": "L3_4",
  "base_center_id": "L3",
  "stage": "old branch line chains",
  "kind": "curve",
  "equations": "a; d",
  "codimension": 2,
  "branch_multiplicity": 2,
  "discrepancy_term": 0,
  "chart_count": 2
},
{
  "order": 11,
  "occurrence_id": "L4_1",
  "base_center_id": "L4",
  "stage": "old branch line chains",
  "kind": "curve",
  "equations": "b; c-d",
  "codimension": 2,
  "branch_multiplicity": 2,
  "discrepancy_term": 0,
  "chart_count": 2
},
{
  "order": 12,
  "occurrence_id": "L4_2",
  "base_center_id": "L4",
  "stage": "old branch line chains",
  "kind": "curve",
  "equations": "b; c-d",
  "codimension": 2,
```

```
    "branch_multiplicity": 2,
    "discrepancy_term": 0,
    "chart_count": 2
  },
  {
    "order": 13,
    "occurrence_id": "L4_3",
    "base_center_id": "L4",
    "stage": "old branch line chains",
    "kind": "curve",
    "equations": "b; c-d",
    "codimension": 2,
    "branch_multiplicity": 2,
    "discrepancy_term": 0,
    "chart_count": 2
  },
  {
    "order": 14,
    "occurrence_id": "L4_4",
    "base_center_id": "L4",
    "stage": "old branch line chains",
    "kind": "curve",
    "equations": "b; c-d",
    "codimension": 2,
    "branch_multiplicity": 2,
    "discrepancy_term": 0,
    "chart_count": 2
  },
  {
    "order": 15,
    "occurrence_id": "L5",
    "base_center_id": "L5",
    "stage": "old branch line chains",
    "kind": "curve",
    "equations": "a; c-d",
    "codimension": 2,
    "branch_multiplicity": 2,
    "discrepancy_term": 0,
    "chart_count": 2
  },
  {
    "order": 16,
    "occurrence_id": "L6_1",
    "base_center_id": "L6",
    "stage": "old branch line chains",
    "kind": "curve",
    "equations": "c; d",
    "codimension": 2,
    "branch_multiplicity": 2,
    "discrepancy_term": 0,
    "chart_count": 2
  },
  {
    "order": 17,
    "occurrence_id": "L6_2",
    "base_center_id": "L6",
    "stage": "old branch line chains",
    "kind": "curve",
    "equations": "c; d",
    "codimension": 2,
    "branch_multiplicity": 2,
    "discrepancy_term": 0,
    "chart_count": 2
  },
  {
    "order": 18,
    "occurrence_id": "L6_3",
    "base_center_id": "L6",
```

```
    "stage": "old branch line chains",
    "kind": "curve",
    "equations": "c; d",
    "codimension": 2,
    "branch_multiplicity": 2,
    "discrepancy_term": 0,
    "chart_count": 2
  },
  {
    "order": 19,
    "occurrence_id": "L6_4",
    "base_center_id": "L6",
    "stage": "old branch line chains",
    "kind": "curve",
    "equations": "c; d",
    "codimension": 2,
    "branch_multiplicity": 2,
    "discrepancy_term": 0,
    "chart_count": 2
  },
  {
    "order": 20,
    "occurrence_id": "L7",
    "base_center_id": "L7",
    "stage": "old branch line chains",
    "kind": "curve",
    "equations": "a-d; b+c-d",
    "codimension": 2,
    "branch_multiplicity": 2,
    "discrepancy_term": 0,
    "chart_count": 2
  },
  {
    "order": 21,
    "occurrence_id": "P13",
    "base_center_id": "P13",
    "stage": "multiplicity-four points",
    "kind": "point",
    "equations": "a; b; d",
    "codimension": 3,
    "branch_multiplicity": 4,
    "discrepancy_term": 0,
    "chart_count": 3
  },
  {
    "order": 22,
    "occurrence_id": "P145",
    "base_center_id": "P145",
    "stage": "multiplicity-four points",
    "kind": "point",
    "equations": "a; b; c-d",
    "codimension": 3,
    "branch_multiplicity": 4,
    "discrepancy_term": 0,
    "chart_count": 3
  },
  {
    "order": 23,
    "occurrence_id": "P27",
    "base_center_id": "P27",
    "stage": "multiplicity-four points",
    "kind": "point",
    "equations": "a-d; b+c-d; c",
    "codimension": 3,
    "branch_multiplicity": 4,
    "discrepancy_term": 0,
    "chart_count": 3
  },
```

```
{
  "order": 24,
  "occurrence_id": "P356",
  "base_center_id": "P356",
  "stage": "multiplicity-four points",
  "kind": "point",
  "equations": "a; c; d",
  "codimension": 3,
  "branch_multiplicity": 4,
  "discrepancy_term": 0,
  "chart_count": 3
},
{
  "order": 25,
  "occurrence_id": "P46",
  "base_center_id": "P46",
  "stage": "multiplicity-four points",
  "kind": "point",
  "equations": "b; c-d; c",
  "codimension": 3,
  "branch_multiplicity": 4,
  "discrepancy_term": 0,
  "chart_count": 3
},
{
  "order": 26,
  "occurrence_id": "P47",
  "base_center_id": "P47",
  "stage": "multiplicity-four points",
  "kind": "point",
  "equations": "b; c-d; a-d",
  "codimension": 3,
  "branch_multiplicity": 4,
  "discrepancy_term": 0,
  "chart_count": 3
},
{
  "order": 27,
  "occurrence_id": "E145_x",
  "base_center_id": "E145_x",
  "stage": "exceptional curves after point blow-ups",
  "kind": "curve",
  "equations": "x; exceptional_145",
  "codimension": 2,
  "branch_multiplicity": 2,
  "discrepancy_term": 0,
  "chart_count": 2
},
{
  "order": 28,
  "occurrence_id": "E145_z",
  "base_center_id": "E145_z",
  "stage": "exceptional curves after point blow-ups",
  "kind": "curve",
  "equations": "z; exceptional_145",
  "codimension": 2,
  "branch_multiplicity": 2,
  "discrepancy_term": 0,
  "chart_count": 2
},
{
  "order": 29,
  "occurrence_id": "E356_x",
  "base_center_id": "E356_x",
  "stage": "exceptional curves after point blow-ups",
  "kind": "curve",
  "equations": "x; exceptional_356",
  "codimension": 2,
```

```
    "branch_multiplicity": 2,
    "discrepancy_term": 0,
    "chart_count": 2
  },
  {
    "order": 30,
    "occurrence_id": "E356_z",
    "base_center_id": "E356_z",
    "stage": "exceptional curves after point blow-ups",
    "kind": "curve",
    "equations": "z; exceptional_356",
    "codimension": 2,
    "branch_multiplicity": 2,
    "discrepancy_term": 0,
    "chart_count": 2
  },
  {
    "order": 31,
    "occurrence_id": "E46_q",
    "base_center_id": "E46_q",
    "stage": "exceptional curves after point blow-ups",
    "kind": "curve",
    "equations": "q; exceptional_46",
    "codimension": 2,
    "branch_multiplicity": 2,
    "discrepancy_term": 0,
    "chart_count": 2
  },
  {
    "order": 32,
    "occurrence_id": "Gamma_46",
    "base_center_id": "Gamma_46",
    "stage": "residual Gamma_46",
    "kind": "curve",
    "equations": "e; (T-1)*(r-e)+T",
    "codimension": 2,
    "branch_multiplicity": 2,
    "discrepancy_term": 0,
    "chart_count": 2
  },
  {
    "order": 33,
    "occurrence_id": "Gamma_46_T1_a",
    "base_center_id": "Gamma_46_T1_a",
    "stage": "Gamma_46 endpoint T=1",
    "kind": "curve",
    "equations": "n; tau",
    "codimension": 2,
    "branch_multiplicity": 2,
    "discrepancy_term": 0,
    "chart_count": 2
  },
  {
    "order": 34,
    "occurrence_id": "Gamma_46_T1_b",
    "base_center_id": "Gamma_46_T1_b",
    "stage": "Gamma_46 endpoint T=1",
    "kind": "curve",
    "equations": "m; tau",
    "codimension": 2,
    "branch_multiplicity": 2,
    "discrepancy_term": 0,
    "chart_count": 2
  },
  {
    "order": 35,
    "occurrence_id": "Gamma_46_Tinf_a",
    "base_center_id": "Gamma_46_Tinf_a",
```

```
      "stage": "Gamma_46 endpoint T=infinity",
      "kind": "curve",
      "equations": "e; K",
      "codimension": 2,
      "branch_multiplicity": 2,
      "discrepancy_term": 0,
      "chart_count": 2
    },
    {
      "order": 36,
      "occurrence_id": "Gamma_46_Tinf_b",
      "base_center_id": "Gamma_46_Tinf_b",
      "stage": "Gamma_46 endpoint T=infinity",
      "kind": "curve",
      "equations": "m; U",
      "codimension": 2,
      "branch_multiplicity": 2,
      "discrepancy_term": 0,
      "chart_count": 2
    }
  ]
}
```

Appendix E

Code Listings

E.1 Crepant Ledger Python

```
#!/usr/bin/env python3
"""Resolution ledger for the Q1 double-octic report.

The only numerical crepantness test used here is

    codim(center) - 1 - floor(branch_multiplicity/2) = 0.

For the Q1 sequence this means curves of branch multiplicity 2 and points of
branch multiplicity 4.  The final four ordinary double points are resolved by
global Weil-divisor blowups on the double cover and are recorded separately.
"""

from __future__ import annotations

import argparse
import csv
import json
from dataclasses import asdict, dataclass
from pathlib import Path


@dataclass(frozen=True)
class Center:
    center_id: str
    stage: str
    kind: str
    equations: tuple[str, ...]
    codimension: int
    branch_multiplicity: int
    repeat: int = 1
    note: str = ""

    @property
    def discrepancy_term(self) -> int:
        return self.codimension - 1 - self.branch_multiplicity // 2

    @property
    def chart_count(self) -> int:
        return len(self.equations) * self.repeat


@dataclass(frozen=True)
class SmallResolution:
    divisor_id: str
    equations: tuple[str, str]
```

```
    nodes: tuple[str, ...]


CENTERS = (
    Center("L1", "old branch line chains", "curve", ("a", "b"), 2, 2, 4, "generic cA8"),
    Center("L2", "old branch line chains", "curve", ("a-b", "c"), 2, 2, 2, "generic cA4"),
    Center("L3", "old branch line chains", "curve", ("a", "d"), 2, 2, 4, "generic cA7"),
    Center("L4", "old branch line chains", "curve", ("b", "c-d"), 2, 2, 4, "generic cA8"),
    Center("L5", "old branch line chains", "curve", ("a", "c-d"), 2, 2, 1, "generic cA1"),
    Center("L6", "old branch line chains", "curve", ("c", "d"), 2, 2, 4, "generic cA7"),
    Center("L7", "old branch line chains", "curve", ("a-d", "b+c-d"), 2, 2, 1, "generic cA1"),
    Center("P13", "multiplicity-four points", "point", ("a", "b", "d"), 3, 4, 1, "[0:0:1:0]"),
    Center("P145", "multiplicity-four points", "point", ("a", "b", "c-d"), 3, 4, 1, "[0:0:1:1]"),
    Center("P27", "multiplicity-four points", "point", ("a-d", "b+c-d", "c"), 3, 4, 1, "[1:0:0:1]"),
    Center("P356", "multiplicity-four points", "point", ("a", "c", "d"), 3, 4, 1, "[0:1:0:0]"),
    Center("P46", "multiplicity-four points", "point", ("b", "c-d", "c"), 3, 4, 1, "[1:0:0:0]"),
    Center("P47", "multiplicity-four points", "point", ("b", "c-d", "a-d"), 3, 4, 1, "[1:0:1:1]"),
    Center("E145_x", "exceptional curves after point blow-ups", "curve", ("x", "exceptional_145"), 2, 2),
    Center("E145_z", "exceptional curves after point blow-ups", "curve", ("z", "exceptional_145"), 2, 2),
    Center("E356_x", "exceptional curves after point blow-ups", "curve", ("x", "exceptional_356"), 2, 2),
    Center("E356_z", "exceptional curves after point blow-ups", "curve", ("z", "exceptional_356"), 2, 2),
    Center("E46_q", "exceptional curves after point blow-ups", "curve", ("q", "exceptional_46"), 2, 2),
    Center("Gamma_46", "residual Gamma_46", "curve", ("e", "(T-1)*(r-e)+T"), 2, 2, 1, "main residual curve"),
    Center("Gamma_46_T1_a", "Gamma_46 endpoint T=1", "curve", ("n", "tau"), 2, 2, 1, "first finite endpoint"),
    Center("Gamma_46_T1_b", "Gamma_46 endpoint T=1", "curve", ("m", "tau"), 2, 2, 1, "second finite endpoint"),

    Center("Gamma_46_Tinf_a", "Gamma_46 endpoint T=infinity", "curve", ("e", "K"), 2, 2, 1, "first infinite
     endpoint"),
    Center("Gamma_46_Tinf_b", "Gamma_46 endpoint T=infinity", "curve", ("m", "U"), 2, 2, 1, "second infinite
     endpoint"),
)

SMALL_RESOLUTIONS = (
    SmallResolution("Dab_plus", ("b-a", "R-a*c*(a^2+c*d-d^2)"), ("N13_0",)),
    SmallResolution("Db_minus", ("b", "R+a*(c-d)*(a*c-2*c*d+d^2)"), ("N13_half", "N46_0")),
    SmallResolution("Dd_minus", ("d", "R+a*c*(a*b+a*c-b*c)"), ("N46_qt",)),
)


def expanded_centers() -> list[dict[str, object]]:
    rows: list[dict[str, object]] = []
    order = 0
    for center in CENTERS:
        for repeat_index in range(1, center.repeat + 1):
            order += 1
            rows.append(
                {
                    "order": order,
                    "occurrence_id": f"{center.center_id}_{repeat_index}" if center.repeat > 1 else center.
     center_id,
                    "base_center_id": center.center_id,
                    "stage": center.stage,
                    "kind": center.kind,
                    "equations": "; ".join(center.equations),
                    "codimension": center.codimension,
                    "branch_multiplicity": center.branch_multiplicity,
                    "discrepancy_term": center.discrepancy_term,
                    "chart_count": len(center.equations),
                }
            )
    return rows


def summary() -> dict[str, object]:
    return {
        "base_center_count": len(CENTERS),
        "divisorial_blowup_count": sum(center.repeat for center in CENTERS),
        "explicit_affine_chart_count": sum(center.chart_count for center in CENTERS),
```

```
        "all_discrepancy_terms": [center.discrepancy_term for center in CENTERS],
        "small_resolution_count": sum(len(sr.nodes) for sr in SMALL_RESOLUTIONS),
        "small_resolution_divisors": [asdict(sr) for sr in SMALL_RESOLUTIONS],
    }


def verify() -> None:
    info = summary()
    assert info["divisorial_blowup_count"] == 36, info
    assert info["explicit_affine_chart_count"] == 78, info
    assert all(term == 0 for term in info["all_discrepancy_terms"]), info
    assert info["small_resolution_count"] == 4, info


def write_csv(path: Path, rows: list[dict[str, object]]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="") as fh:
        writer = csv.DictWriter(fh, fieldnames=list(rows[0].keys()))
        writer.writeheader()
        writer.writerows(rows)


def tex_escape(text: object) -> str:
    s = str(text)
    replacements = {
        "\\": r"\textbackslash{}",
        "_": r"\_",
        "&": r"\&",
        "%": r"\%",
        "#": r"\#",
        "{": r"\{",
        "}": r"\}",
    }
    for old, new in replacements.items():
        s = s.replace(old, new)
    return s


def write_base_centers_tex(path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    lines = [
        r"\begin{longtable}{@{}p{0.11\textwidth}p{0.24\textwidth}p{0.08\textwidth}p{0.24\textwidth}rrrr@{}}",
        r"\caption{Base resolution centers for Q1. Repeated centers represent ordered cA-chain blow-ups.}\
     label{tab:base-centers}\\",
        r"\toprule",
        r"Center & Stage & Kind & Equations & Repeat & Codim. & Mult. & Disc. \\",
        r"\midrule",
        r"\endfirsthead",
        r"\toprule Center & Stage & Kind & Equations & Repeat & Codim. & Mult. & Disc. \\ \midrule",
        r"\endhead",
    ]
    for c in CENTERS:
        lines.append(
            r"\texttt{%s} & %s & %s & \(\langle %s\rangle\) & %d & %d & %d & %d \\"
            % (
                tex_escape(c.center_id),
                tex_escape(c.stage),
                tex_escape(c.kind),
                tex_escape(", ".join(c.equations)),
                c.repeat,
                c.codimension,
                c.branch_multiplicity,
                c.discrepancy_term,
            )
        )
    lines.extend([r"\bottomrule", r"\end{longtable}", ""])
    path.write_text("\n".join(lines))
```

```
def write_expanded_centers_tex(path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    rows = expanded_centers()
    lines = [
        r"\begin{longtable}{@{}rp{0.16\textwidth}p{0.18\textwidth}p{0.29\textwidth}rrrr@{}}",
        r"\caption{Expanded ordered list of the 36 divisorial crepant blow-ups.}\label{tab:expanded-centers}\\
    ",
        r"\toprule",
        r"Order & Occurrence & Stage & Equations & Codim. & Mult. & Disc. & Charts \\",
        r"\midrule",
        r"\endfirsthead",
        r"\toprule Order & Occurrence & Stage & Equations & Codim. & Mult. & Disc. & Charts \\ \midrule",
        r"\endhead",
    ]
    for row in rows:
        lines.append(
            r"%d & \texttt{%s} & %s & \(\langle %s\rangle\) & %d & %d & %d & %d \\"
            % (
                row["order"],
                tex_escape(row["occurrence_id"]),
                tex_escape(row["stage"]),
                tex_escape(row["equations"]),
                row["codimension"],
                row["branch_multiplicity"],
                row["discrepancy_term"],
                row["chart_count"],
            )
        )
    lines.extend([r"\bottomrule", r"\end{longtable}", ""])
    path.write_text("\n".join(lines))


def write_small_resolutions_tex(path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    lines = [
        r"\begin{longtable}{@{}p{0.16\textwidth}p{0.45\textwidth}p{0.25\textwidth}@{}}",
        r"\caption{Global Weil divisors used for the projective small resolutions.}\label{tab:small-
    resolutions}\\",
        r"\toprule",
        r"Divisor & Equations & Nodes resolved \\",
        r"\midrule",
        r"\endfirsthead",
        r"\toprule Divisor & Equations & Nodes resolved \\ \midrule",
        r"\endhead",
    ]
    for sr in SMALL_RESOLUTIONS:
        lines.append(
            r"\texttt{%s} & \(\langle %s\rangle\) & %s \\"
            % (
                tex_escape(sr.divisor_id),
                tex_escape(", ".join(sr.equations)),
                tex_escape(", ".join(sr.nodes)),
            )
        )
    lines.extend([r"\bottomrule", r"\end{longtable}", ""])
    path.write_text("\n".join(lines))


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--csv", type=Path)
    parser.add_argument("--json", type=Path)
    parser.add_argument("--base-tex", type=Path)
    parser.add_argument("--expanded-tex", type=Path)
    parser.add_argument("--small-tex", type=Path)
    args = parser.parse_args()
```

```
    verify()
    rows = expanded_centers()
    info = summary()
    print(json.dumps(info, indent=2))

    if args.csv:
        write_csv(args.csv, rows)
    if args.json:
        args.json.parent.mkdir(parents=True, exist_ok=True)
        args.json.write_text(json.dumps({"summary": info, "expanded_centers": rows}, indent=2) + "\n")
    if args.base_tex:
        write_base_centers_tex(args.base_tex)
    if args.expanded_tex:
        write_expanded_centers_tex(args.expanded_tex)
    if args.small_tex:
        write_small_resolutions_tex(args.small_tex)


if __name__ == "__main__":
    main()
```

E.2 Point-Count Python

```
#!/usr/bin/env python3
"""Finite-field point counts for the Q1 double-octic model.

The singular model is

    Y0: R^2 = Delta(a,b,c,d) in P(1,1,1,1,4),
    Delta = q4^2 - 4*q3*q5.

For an odd prime p, each point of P^3(F_p) contributes 1, 2, or 0 points to
the double cover according as Delta is zero, a nonzero square, or a nonsquare.
The crepant model count is then obtained from the resolution ledger:

    #X(F_p) = #Y0(F_p) + 58*p^2 + 82*p.

The script intentionally uses the compact q3/q4/q5 formula instead of the
expanded octic. This keeps the count auditable and avoids transcription
errors in the 50-term expanded polynomial.
"""

from __future__ import annotations

import argparse
import csv
import json
from pathlib import Path


BAD_PRIMES = {2, 3, 13}
PICARD_RANK = 66
DEFAULT_PRIMES = (3, 5, 7, 11, 107)

# These rows are the known values from the checked Q1 ledger.  They are used as
# regression tests, not as inputs to the count.
EXPECTED_SINGULAR_COUNTS = {
    3: 62,
    5: 253,
    7: 637,
    11: 2150,
    107: 1315608,
}
EXPECTED_TRACES = {
```

```
    3: -10,
    5: -7,
    7: -13,
    11: -26,
    107: -684,
}


def q3(a: int, b: int, c: int, d: int, p: int) -> int:
    return (a * d * (b + c - d)) % p


def q4(a: int, b: int, c: int, d: int, p: int) -> int:
    a2, b2, c2, d2 = a * a, b * b, c * c, d * d
    return (
        a2 * b * c
        + a2 * b * d
        + a2 * c2
        - a2 * c * d
        + a * b2 * d
        - a * b * c2
        + a * b * c * d
        - 2 * a * b * d2
        - 2 * a * c2 * d
        + 3 * a * c * d2
        - a * d2 * d
        + b * c2 * d
        - 2 * b * c * d2
        + b * d2 * d
    ) % p


def q5(a: int, b: int, c: int, d: int, p: int) -> int:
    a2, b2, c2, d2 = a * a, b * b, c * c, d * d
    a3 = a2 * a
    return (
        a3 * b * c
        + a2 * b2 * d
        - a2 * b * c2
        - a2 * b * c * d
        - a * b2 * d2
        + 2 * a * b * c * d2
        - a * b * d2 * d
        - b2 * c * d2
        + b2 * d2 * d
    ) % p


def delta(a: int, b: int, c: int, d: int, p: int) -> int:
    q_3 = q3(a, b, c, d, p)
    q_4 = q4(a, b, c, d, p)
    q_5 = q5(a, b, c, d, p)
    return (q_4 * q_4 - 4 * q_3 * q_5) % p


def normalized_projective_points_p3(p: int):
    """Yield one representative for each F_p-point of P^3."""
    for b in range(p):
        for c in range(p):
            for d in range(p):
                yield (1, b, c, d)
    for c in range(p):
        for d in range(p):
            yield (0, 1, c, d)
    for d in range(p):
        yield (0, 0, 1, d)
    yield (0, 0, 0, 1)
```

```
def square_root_count_table(p: int) -> list[int]:
    table = [0] * p
    for x in range(p):
        table[(x * x) % p] += 1
    return table


def count_singular_model(p: int) -> int:
    if p < 2:
        raise ValueError("p must be prime")
    if p == 2:
        return sum(
            sum(1 for r in range(p) if (r * r - delta(a, b, c, d, p)) % p == 0)
            for a, b, c, d in normalized_projective_points_p3(p)
        )
    root_counts = square_root_count_table(p)
    return sum(root_counts[delta(a, b, c, d, p)] for a, b, c, d in normalized_projective_points_p3(p))


def crepant_correction(p: int) -> int:
    return 58 * p * p + 82 * p


def resolved_count(p: int, singular_count: int) -> int:
    return singular_count + crepant_correction(p)


def trace_from_count(p: int, count_x: int) -> int:
    return 1 + p**3 + PICARD_RANK * (p * p + p) - count_x


def compute_row(p: int) -> dict[str, int | bool]:
    singular = count_singular_model(p)
    correction = crepant_correction(p)
    resolved = resolved_count(p, singular)
    trace = trace_from_count(p, resolved)
    return {
        "p": p,
        "singular_model_count": singular,
        "crepant_correction": correction,
        "resolved_count": resolved,
        "trace_a_p": trace,
        "good_prime": p not in BAD_PRIMES,
    }


def verify_rows(rows: list[dict[str, int | bool]]) -> None:
    for row in rows:
        p = int(row["p"])
        if p in EXPECTED_SINGULAR_COUNTS:
            assert row["singular_model_count"] == EXPECTED_SINGULAR_COUNTS[p], row
        if p in EXPECTED_TRACES:
            assert row["trace_a_p"] == EXPECTED_TRACES[p], row


def write_csv(path: Path, rows: list[dict[str, int | bool]]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="") as fh:
        writer = csv.DictWriter(fh, fieldnames=list(rows[0].keys()))
        writer.writeheader()
        writer.writerows(rows)


def write_json(path: Path, rows: list[dict[str, int | bool]]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(rows, indent=2) + "\n")
```

```
def write_tex(path: Path, rows: list[dict[str, int | bool]]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    lines = [
        r"\begin{longtable}{@{}rrrrrl@{}}",
        r"\caption{Finite-field counts for the Q1 singular double octic and its crepant model.}\label{tab:
     point-counts}\\",
        r"\toprule",
        r"\(p\) & \(\#Y_0(\mathbb F_p)\) & \(58p^2+82p\) & \(\#X(\mathbb F_p)\) & \(a_p(X)\) & Use \\",
        r"\midrule",
        r"\endfirsthead",
        r"\toprule \(p\) & \(\#Y_0(\mathbb F_p)\) & \(58p^2+82p\) & \(\#X(\mathbb F_p)\) & \(a_p(X)\) & Use \\
      \midrule",
        r"\endhead",
    ]
    for row in rows:
        use = "good" if row["good_prime"] else "bad"
        lines.append(
            r"%d & %d & %d & %d & %d & %s \\"
            % (
                row["p"],
                row["singular_model_count"],
                row["crepant_correction"],
                row["resolved_count"],
                row["trace_a_p"],
                use,
            )
        )
    lines.extend([r"\bottomrule", r"\end{longtable}", ""])
    path.write_text("\n".join(lines))


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("primes", nargs="*", type=int, default=list(DEFAULT_PRIMES))
    parser.add_argument("--csv", type=Path)
    parser.add_argument("--json", type=Path)
    parser.add_argument("--tex", type=Path)
    args = parser.parse_args()

    rows = [compute_row(p) for p in args.primes]
    verify_rows(rows)

    for row in rows:
        print(
            "p={p:>3}  #Y0={singular_model_count:>8}  correction={crepant_correction:>8}  "
            "#X={resolved_count:>8}  a_p={trace_a_p:>5}  good={good_prime}".format(**row)
        )

    if args.csv:
        write_csv(args.csv, rows)
    if args.json:
        write_json(args.json, rows)
    if args.tex:
        write_tex(args.tex, rows)


if __name__ == "__main__":
    main()
```

E.3 Model Checks in Macaulay2

```
-- Macaulay2 checks for the Q1 double-octic model used in the report.
-- Run from the repository root:
```

```
--
--     M2 --script report/code/q1_model_checks.m2
--
-- The script records the projection model, decomposes the singular locus of the
-- branch octic, and checks that the three global divisors used for the final
-- small resolutions lie on the double cover.

R = QQ[a,b,c,d]

q3 = a*d*(b+c-d)
q4 = a^2*b*c+a^2*b*d+a^2*c^2-a^2*c*d+a*b^2*d-a*b*c^2+a*b*c*d-2*a*b*d^2-2*a*c^2*d+3*a*c*d^2-a*d^3+b*c^2*d-2*b*c
     *d^2+b*d^3
q5 = a^3*b*c+a^2*b^2*d-a^2*b*c^2-a^2*b*c*d-a*b^2*d^2+2*a*b*c*d^2-a*b*d^3-b^2*c*d^2+b^2*d^3
Delta = q4^2 - 4*q3*q5

irr = ideal(a,b,c,d)
singularIdeal = saturate(ideal(Delta) + ideal(diff(a,Delta), diff(b,Delta), diff(c,Delta), diff(d,Delta)), irr
     )
branchComponents = decompose singularIdeal

print("Q1_BRANCH_COMPONENT_COUNT|" | toString(#branchComponents))
scan(#branchComponents, i -> (
    J := branchComponents#i;
    print("Q1_BRANCH_COMPONENT|" | toString(i) | "|projective_dim=" | toString(dim(R/J)-1) | "|degree=" |
     toString(degree J) | "|gens=" | toString(mingens J))
))

S = QQ[a,b,c,d,Rcov]
DeltaS = sub(Delta, S)
coverEq = Rcov^2 - DeltaS

DabPlus = ideal(b-a, Rcov-a*c*(a^2+c*d-d^2))
DbMinus = ideal(b, Rcov+a*(c-d)*(a*c-2*c*d+d^2))
DdMinus = ideal(d, Rcov+a*c*(a*b+a*c-b*c))

print("Dab_plus_lies_on_cover|" | toString(isSubset(ideal coverEq, DabPlus)))
print("Db_minus_lies_on_cover|" | toString(isSubset(ideal coverEq, DbMinus)))
print("Dd_minus_lies_on_cover|" | toString(isSubset(ideal coverEq, DdMinus)))

-- The repository K3_Fibration octic is intentionally not checked here.  The
-- comparison script in the source tree,
-- codexCrepeantResolutions/q1_compare_models.m2, shows that its singular-locus
-- signature differs from the paper model used in this report.
```

E.4 Repository Comparison in Macaulay2

```
-- Compare the paper Q1 branch octic with the older repository octic from
-- data/Q1/K3_Fibration.m2.  Run from the repository root:
--
--     M2 --script report/code/q1_repository_comparison.m2
--
-- A false signature equality means the K3_Fibration octic cannot be used as a
-- projective-isomorphic substitute for the paper double-octic model.

load "codexCrepeantResolutions/q1_paper_model.m2"

paperComponents = singularComponents()
print("PAPER_COMPONENTS|" | toString(#paperComponents))
scan(#paperComponents, i -> (
    J := paperComponents#i;
    print("PAPER|" | toString(i) | "|" | toString(dim(R/J)-1) | "|" | toString(degree J) | "|" | toString(
     mingens J))
))
```

```
S = QQ[y,z,w,v]
Drepo = y^4*z^2*w^2-2*y^2*z^4*w^2+z^6*w^2+4*y^2*z^3*w^3-4*z^5*w^3-2*y^2*z^2*w^4+6*z^4*w^4-4*z^3*w^5+z^2*w^6-4*
     y^4*z^2*w*v+6*y^3*z^3*w*v-2*y^2*z^4*w*v+2*y*z^5*w*v-2*z^6*w*v-6*y^3*z^2*w^2*v-2*y*z^4*w^2*v+8*z^5*w^2*v+2*
     y^2*z^2*w^3*v-2*y*z^3*w^3*v-12*z^4*w^3*v+2*y*z^2*w^4*v+8*z^3*w^4*v-2*z^2*w^5*v+4*y^4*z^2*v^2-8*y^3*z^3*v
     ^2+5*y^2*z^4*v^2-2*y*z^5*v^2+z^6*v^2+2*y^4*z*w*v^2+4*y^3*z^2*w*v^2-2*y*z^4*w*v^2-4*z^5*w*v^2+4*y^3*z*w^2*
     v^2-7*y^2*z^2*w^2*v^2+10*y*z^3*w^2*v^2+6*z^4*w^2*v^2+2*y^2*z*w^3*v^2-6*y*z^2*w^3*v^2-4*z^3*w^3*v^2+z^2*w
     ^4*v^2-4*y^4*z*v^3+6*y^3*z^2*v^3-6*y^2*z^3*v^3+4*y*z^4*v^3-6*y^3*z*w*v^3+8*y^2*z^2*w*v^3-8*y*z^3*w*v^3-2*
     y^2*z*w^2*v^3+4*y*z^2*w^2*v^3+y^4*v^4
repoSing = saturate(ideal(Drepo) + ideal(diff(y,Drepo), diff(z,Drepo), diff(w,Drepo), diff(v,Drepo)), ideal(y,
     z,w,v))
repoComponents = decompose repoSing

print("REPO_COMPONENTS|" | toString(#repoComponents))
scan(#repoComponents, i -> (
    J := repoComponents#i;
    print("REPO|" | toString(i) | "|" | toString(dim(S/J)-1) | "|" | toString(degree J) | "|" | toString(
     mingens J))
))

paperSignature = sort apply(paperComponents, J -> (dim(R/J)-1, degree J))
repoSignature = sort apply(repoComponents, J -> (dim(S/J)-1, degree J))
print("SIGNATURE_EQUAL|" | toString(paperSignature == repoSignature))
```

Bibliography

[1] Michael F. Atiyah. On analytic surfaces with double points. *Proceedings of the Royal Society of London. Series A*, 247(1249):237–244, 1958. doi: 10.1098/rspa.1958.0181.

[2] Slawomir Cynk. Double coverings of octic arrangements with isolated singularities, 1999. URL https://arxiv.org/abs/math/9902077.

[3] Slawomir Cynk and Christian Meyer. Geometry and arithmetic of certain double octic calabi–yau manifolds. *Canadian Mathematical Bulletin*, 48(2):180–194, 2005. doi: 10.4153/CMB-2005-016-3. URL https://arxiv.org/abs/math/0304121.

[4] Slawomir Cynk and Tomasz Szemberg. Double covers of $\mathbb{P}^3$ and calabi–yau varieties, 1999. URL https://arxiv.org/abs/math/9902057.

[5] Slawomir Cynk and Duco van Straten. Infinitesimal deformations of double covers of smooth algebraic varieties, 2003. URL https://arxiv.org/abs/math/0303329.

[6] Gerd Faltings. Endlichkeitssätze für abelsche varietäten über zahlkörpern. *Inventiones Mathematicae*, 73:349–366, 1983. doi: 10.1007/BF01388432.

[7] Colin Ingalls and Adam Logan. On the cynk–hulek criterion for crepant resolutions of double covers, 2020. URL https://arxiv.org/abs/2006.14981.

[8] Viktor S. Kulikov. Degenerations of K3 surfaces and Enriques surfaces. *Mathematics of the USSR-Izvestiya*, 11(5):957–989, 1977. doi: 10.1070/IM1977v011n05ABEH001753.

[9] Ron Livné. Motivic orthogonal two-dimensional representations of $\mathrm{Gal}(\overline{\mathbb{Q}}/\mathbb{Q})$. *Israel Journal of Mathematics*, 92:149–156, 1995.

[10] Adam Logan. New realizations of modular forms in calabi–yau threefolds arising from ϕ^4 theory. *Journal of Number Theory*, 184:342–383, 2018. doi: 10.1016/j.jnt.2017.09.001. URL https://arxiv.org/abs/1604.04918.

[11] Jean-Pierre Serre. *Abelian l-adic Representations and Elliptic Curves*. W. A. Benjamin, New York, 1968.

[12] The LMFDB Collaboration. The l-functions and modular forms database: Newform 13.4.a.a. https://www.lmfdb.org/ModularForm/GL2/Q/holomorphic/13/4/a/a/, 2026. Accessed July 13, 2026.